



A parallel approach for solving the Green's function based on the nested-dissection and selected-inverse methods

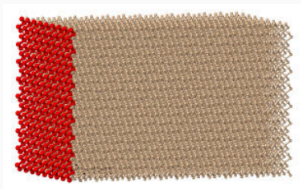
Emmanuel Agullo, Alfredo Buttari, Matthieu Robeyns

Berlin, March 3, 2026, MS13 at SIAM PP26

Collaboration within EuroHPC EoCoE-III project between :

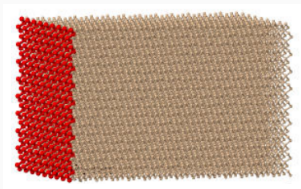
- **France** : CNRS+Inria-Concace team : Emmanuel Agullo, Alfredo Buttari, Matthieu Robeyns.
- **Germany** : Jülich team : Edoardo Di Napoli, Christoph Conrads, and Gustavo Ramirez Hidalgo.
- **Italy** : CNR team : Alessandro Pecchia, and Flavia Carafone.

Study case : Non-Equilibrium Green's Function (NEGF) equation – Dyson equation



- Develop application on **quantum transport** with new circuits (**more challenging regarding time and memory consumption**).

Study case : Non-Equilibrium Green's Function (NEGF) equation – Dyson equation



- Develop application on **quantum transport** with new circuits (**more challenging regarding time and memory consumption**).
- Dyson equation :

$$[ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I \Leftrightarrow T(k, E)G^r(k, E) = I$$

Study case : Non-Equilibrium Green's Function (NEGF) equation – Dyson equation



Quantum circuit (left) represented as tridiagonal matrix T (right).

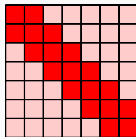
- Develop application on **quantum transport** with new circuits (**more challenging regarding time and memory consumption**).
- Dyson equation :

$$[ES(k) - H(k) - \Sigma^r(k, E)]G^r(k, E) = I \Leftrightarrow T(k, E)G^r(k, E) = I$$

- Consider $T(k, E)$ is a block tridiagonal matrix.
- $G^r(k, E) = T^{-1}(k, E)$?

Dyson equation : Find $G^r(k, e)$

 Compute $T^{-1}(k, E)$, because $T^{-1}(k, E)$ is dense.



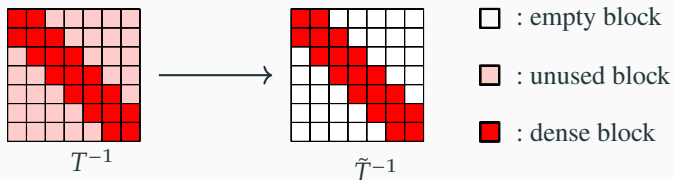
T^{-1}

 : unused block

 : dense block

Dyson equation : Find $G^r(k, e)$

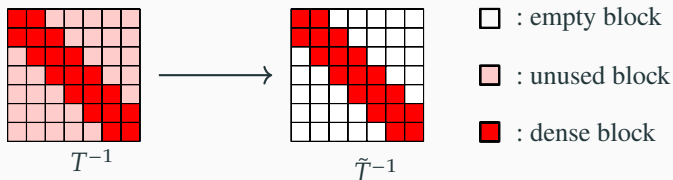
👎 Compute $T^{-1}(k, E)$, because $T^{-1}(k, E)$ is dense.



👍 The application only needs tridiagonals block of $T^{-1}(k, E)$.

Dyson equation : Find $G^r(k, e)$

👎 Compute $T^{-1}(k, E)$, because $T^{-1}(k, E)$ is dense.



👍 The application only needs tridiagonals block of $T^{-1}(k, E)$.

👍 Can avoid off-diagonal computation $\Rightarrow$ related work : Block Recursive¹, Block-Cyclic Reduction².

- Develop a **new framework** with parallelism that takes the **Selected Inverse³(SI)** algorithm.

1. Recursive Block Coding - A New Approach to Transform Coding, Farrelle et Jain, 1986
2. Cyclic reduction-history and applications, Gander et Golub, 1998
3. SelInv—An Algorithm for Selected Inversion of a Sparse Symmetric Matrix, 2011

Selected Inverse^a

^aSelInv—An Algorithm for Selected Inversion of a Sparse Symmetric Matrix, 2011

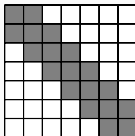
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$                                      // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$                        // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$                      // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$                      // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block



Factorization : Algorithm

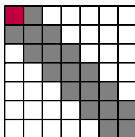
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$  // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$  // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$  // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$  // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block



Factorization : Algorithm

Algorithm : LU factorization of T

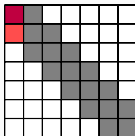
Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$  // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$  // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$  // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$  // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block

■ : L factor block



Factorization : Algorithm

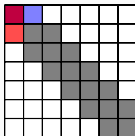
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$  // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$  // Step 2 : Update  $L_{i+1:nb,i}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$  // Step 3 : Update  $U_{i,i+1:nb}$ 
7    $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$  // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block



Factorization : Algorithm

Algorithm : LU factorization of T

Input : $T::\text{Matrix}$

Output : $T::\text{Matrix}$ (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
```

```

2  for  $i$  in  $1 : n-1$  do

```

3 $T_{i,i} \leftarrow \text{lu}(T_{i,i})$

```
// Step 1 : create  $L_{i,i}U_{i,i}$ 
```

4	if $T_{i,i+1} \neq 0$ then
---	--

```
// Step 2 : Update  $L_{i+1:nb,i}$ 
```

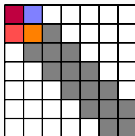
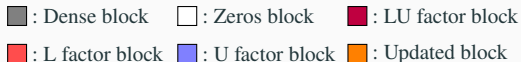
$$5 \quad T_{i+1,i} \leftarrow T_{i+1,i} U_{i,i}^{-1}$$

```
// Step 3 : Update  $U_{i,i+1:nb}$ 
```

6		$T_{i,i+1} \leftarrow L_{i,i}^{-1} T_{i,i+1}$
---	--	---

$$7 \quad T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$$

```
// Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```



Factorization : Algorithm

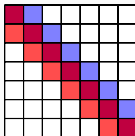
Algorithm : LU factorization of T

Input : T ::Matrix

Output : T ::Matrix (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$  // Step 1 : create  $L_{i,i}U_{i,i}$ 
4   if  $T_{i,i+1} \neq 0$  then // Step 2 : Update  $L_{i+1:nb,i}$ 
5      $T_{i+1,i} \leftarrow T_{i+1,i}U_{i,i}^{-1}$  // Step 3 : Update  $U_{i,i+1:nb}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1}T_{i,i+1}$ 
7      $T_{i+1,i+1} \leftarrow -T_{i+1,i}T_{i,i+1}$  // Step 4 : Update  $T_{i+1:nb,i+1:nb}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block



Let $T \in \mathbb{R}^{m \times m}$, n diagonal blocks of size q .

- **Cost** : $\mathcal{O}(n(\frac{11}{3}q^3 + q^2))$
- **Memory** : $\mathcal{O}((3n - 2)q^2)$

Selected Inverse : Algorithm

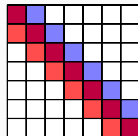
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$            // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$        // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block



Selected Inverse : Algorithm

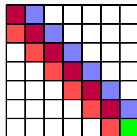
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$  // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$  // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Selected Inverse : Algorithm

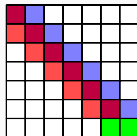
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$  // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$  // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Selected Inverse : Algorithm

Algorithm : Selected inverse of T

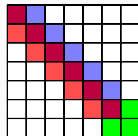
Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$ 
```

// Step 1 : Compute $T_{i+1,i}$
// Step 2 : Compute $T_{i,i+1}$
// Step 3 : Update $T_{i,i}$

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Selected Inverse : Algorithm

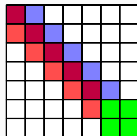
Algorithm : Selected inverse of T

Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$  // Step 1 : Compute  $T_{i+1,i}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$  // Step 2 : Compute  $T_{i,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$  // Step 3 : Update  $T_{i,i}$ 
```

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Selected Inverse : Algorithm

Algorithm : Selected inverse of T

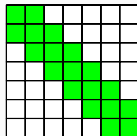
Input : T ::Matrix (in LU format)

Output : T ::Matrix (in inverse format)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $n : -1 : 1$  do
3    $T_{i+1,i} \leftarrow -T_{i+1,i+1}T_{i+1,i}L_{i,i}^{-1}$ 
4    $T_{i,i+1} \leftarrow -U_{i,i}^{-1}T_{i,i+1}T_{i+1,i+1}$ 
5    $T_{i,i} \leftarrow U_{i,i}^{-1}L_{i,i}^{-1} + T_{i,i+1}L_{i+1,i}^{-1}$ 
```

// Step 1 : Compute $T_{i+1,i}$
// Step 2 : Compute $T_{i,i+1}$
// Step 3 : Update $T_{i,i}$

■ : Dense block □ : Zeros block ■ : LU factor block
■ : L factor block ■ : U factor block ■ : Updated block
■ : Inversed block



Let $T \in \mathbb{R}^{m \times m}$, n diagonal blocks of size q .

- **Cost :** $\mathcal{O}(n(4q^3 + q^2) - (q^3 + q^2))$
- **Memory :** $\mathcal{O}((3n - 2)q^2)$

- LU+SI can be easily generalized for any type of matrix.

- LU+SI can be easily generalized for any type of matrix.

- LU : **Fill-in**^{4 5}.

■ : Dense block

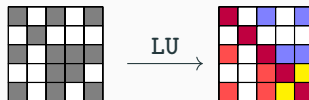
□ : Zeros block

■ : LU factor block

■ : L factor block

■ : U factor block

■ : Fill-in block



4. Direct methods, Duff, 1998

5. A survey of direct methods for sparse linear systems, Davis, Rajamanickam, et Sid-Lakhdar, 2016

- LU+SI can be easily generalized for any type of matrix.

- LU : **Fill-in**^{4 5}.

■ : Dense block

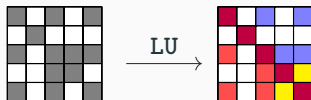
□ : Zeros block

■ : LU factor block

■ : L factor block

■ : U factor block

■ : Fill-in block



- SI : No additional fill-in ; but keeps same structure as LU factors.

4. Direct methods, Duff, 1998

5. A survey of direct methods for sparse linear systems, Davis, Rajamanickam, et Sid-Lakhdar, 2016

- LU+SI can be easily generalized for any type of matrix.

- LU : **Fill-in**^{4 5}.

■ : Dense block

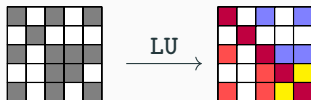
□ : Zeros block

■ : LU factor block

■ : L factor block

■ : U factor block

■ : Fill-in block



- SI : No additional fill-in ; but keeps same structure as LU factors.



Convenient framework to investigate **parallelism**.

4. Direct methods, Duff, 1998

5. A survey of direct methods for sparse linear systems, Davis, Rajamanickam, et Sid-Lakhdar, 2016

Parallelism

How to extract parallelism

 : Dense block

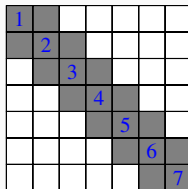
 : Zeros block

1						
	2					
		3				
			4			
				5		
					6	
						7

How to extract parallelism

■ : Dense block

□ : Zeros block



elimination tree⁶

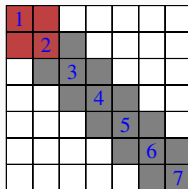


6. The role of elimination trees in sparse factorization, Liu, J. W., 1990

How to extract parallelism

■ : Dense block

□ : Zeros block



elimination tree⁶

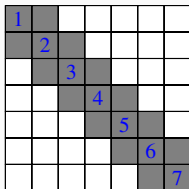


6. The role of elimination trees in sparse factorization, Liu, J. W., 1990

How to extract parallelism

■ : Dense block

□ : Zeros block



elimination tree⁶



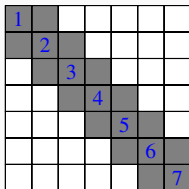
- Computation order :
 - LU : Block 1 to 7.

6. The role of elimination trees in sparse factorization, Liu, J. W., 1990

How to extract parallelism

■ : Dense block

□ : Zeros block



elimination tree⁶



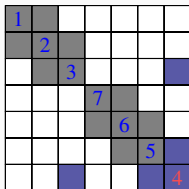
- Computation order :
 - LU : Block 1 to 7.
 - SI : Block 7 to 1.

6. The role of elimination trees in sparse factorization, Liu, J. W., 1990

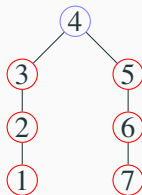
How to extract parallelism

 : Dense block

 : Zeros block



elimination tree⁶



- Computation order :

- LU : Block 1 to 4.
- SI : Block 4 to 1.



Branch 1 – 2 – 3 and 7 – 6 – 5 can be done in parallel !

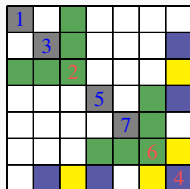
6. The role of elimination trees in sparse factorization, Liu, J. W., 1990

Nested-dissection

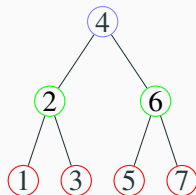
 : Dense block

 : Fill-in block

 : Zeros block



dependency's graph



- **Nested-dissection**⁷ : Recursively reordering the matrix.



Increase **parallelism**.



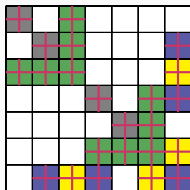
Add **fill-in** effect.

Nested-dissection + Block splitting

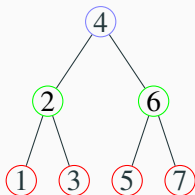
 : Dense block

 : Fill-in block

 : Zeros block



dependency's graph
→



- **Nested-dissection**⁷ : Recursively reordering the matrix.



Increase **parallelism**.



Add **fill-in** effect.

- **Block splitting** : split a block into multiple tiles.



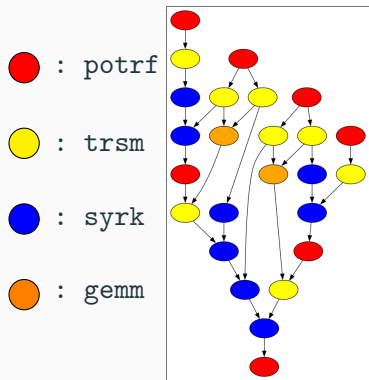
Increase **parallelism** (no additional fill-in).



Small **granularity**.

Find the *good trade-off* between **parallelization**, **fill-in**, and **granularity**.

Implementation



DAG of LU ; two level of ND ($l=2$).

- Consider block computation as **task**.
- Represented as **Directed Acyclic Graph (DAG)**, where :
 - nodes = task (elementary operation),
 - edges = dependencies.
- Task execute when they have no dependency with other task.



Execution of **independent** task can be done in parallel.

Sequential task flow

- **Sequential task flow (STF)** : automatically generate the DAG of task and schedule task to be execute at the runtime.
 - Dependency control are automatically manage by the STF library.
-  Easy to implement. Minor modification from sequential code.

Algorithm : Sequential factorization

Input : $T::\text{Matrix}$

Output : $T::\text{Matrix}$ (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3    $T_{i,i} \leftarrow \text{lu}(T_{i,i})$ 
4   for  $j$  in  $i+1 : n$  when  $T_{i,j} \neq 0$  do
5      $T_{i+1,i} \leftarrow T_{i+1,i} U_{i,i}^{-1}$ 
6      $T_{i,i+1} \leftarrow L_{i,i}^{-1} T_{i,i+1}$ 
7   for  $k$  in index of intersect  $L_{i+1:n,i}, U_{i,i+1:n}$  do
8      $T_k \leftarrow -T_{k[1],i} T_{i,k[2]}$ 
```

Algorithm : Parallel factorization

Input : $T::\text{Matrix}$

Output : $T::\text{Matrix}$ (factor form)

```
1  $n \leftarrow \text{size}(T, 1)$ 
2 for  $i$  in  $1 : n-1$  do
3   INSERT_TASK(InOut( $T_{i,i}$ ), potrf)
4   for  $j$  in  $i+1 : n$  when  $T_{i,j} \neq 0$  do
5     INSERT_TASK(InOut( $T_{j,i}$ ), In( $U_{i,i}$ ), ltrsm)
6     INSERT_TASK(InOut( $T_{i,j}$ ), In( $L_{i,i}$ ), rtrsm)
7   for  $k$  in index of intersect  $L_{i+1:n,i}, U_{i,i+1:n}$  do
8     INSERT_TASK(InOut( $T_k$ ), In( $T_{k[1],i}, T_{i,k[2]}$ ), gemm)
```

Experiments

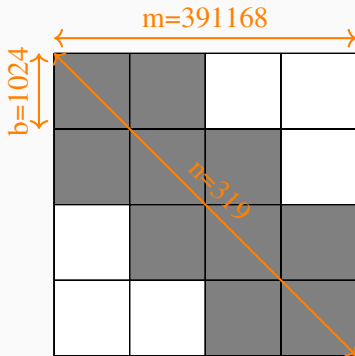
Experimental setup

Hardware/Software :

- Numerical library : Implementation on Chameleon⁸ (in C).
- Task-based/STF runtime : StarPU⁹.
- Hardware (PlafRIM¹⁰) : 64-core AMD Zen2.
- Kernel : SI (DGETRI_ND).

Dataset :

- $n = 319$: Number of diagonal blocks.
- $b = 1,024$: Block size.
- $m = 391,168$: Matrix size.
- $p = \{1, 2, 4, \dots, 64\}$: Number of threads.
- $l = \{0, 1, \dots, 6\}$: Level of nested dissection.
- $s = \{0, 2, 4\}$: Level of tile splitting.

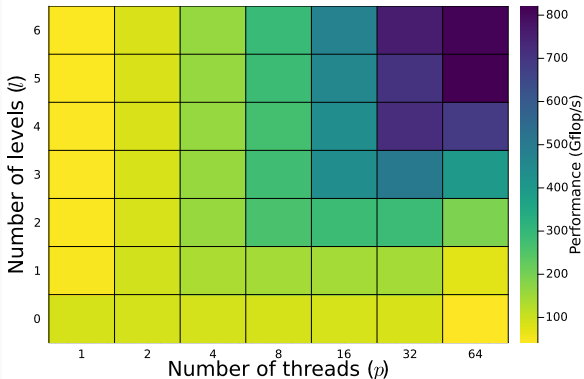
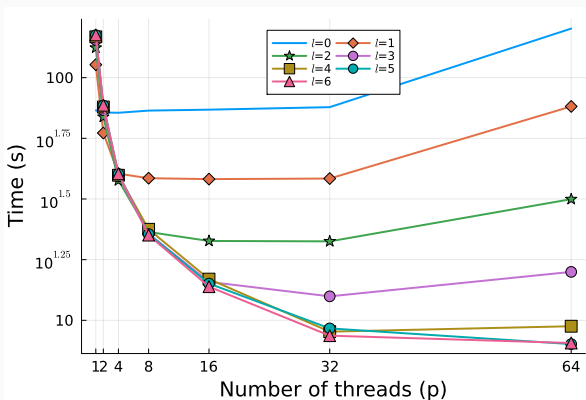


8. <https://solverstack.gitlabpages.inria.fr/chameleon/>

9. <https://starpu.gitlabpages.inria.fr/>

10. <https://www.plafrim.fr/>

Time (s) & Performance (Gflop/s) : Impact of ND (l) (no block splitting)

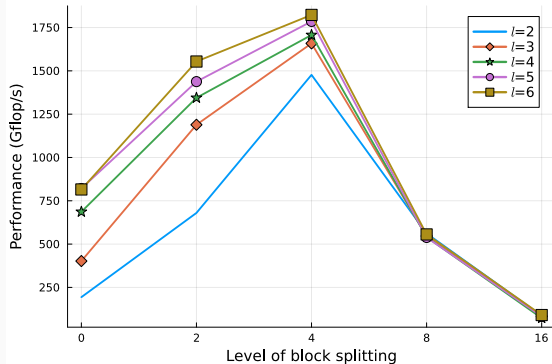
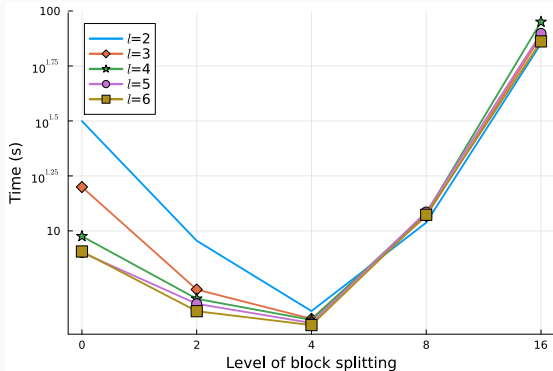


Good scaling between l and p .



If $p > l$ loose in performance.

Impact of ND (l) with block splitting (s) ; threads (p) = 64



- 👍 l with one more level of s is faster than $l - 1$.
- 👍 Tasks' pipelining increase the performance.
- 👎 If s high, bad performance (too small blocks).

1. Parallel implementation of the recursive Green's function method, Drouvelis, Panagiotis S and Schmelcher, Peter and Bastian, Peter, Journal of Computational Physics, 215, 2, 741–756, 2006, Elsevier.
2. Nested dissection solver for transport in 3D nano-electronic devices, Zhao, Y and Hetmaniuk, U and Patil, SR and Qi, J and Anantram, MP, Journal of Computational Electronics, 15, 2, 708–720, 2016, Springer.
3. Accelerated cyclic reduction : a distributed-memory fast solver for structured linear systems, Chávez, Gustavo and Turkiyyah, George and Zampini, Stefano and Ltaief, Hatem and Keyes, David, Parallel Computing, 74, 65–83, 2018, Elsevier.
4. GPU-accelerated parallel selected inversion for structured matrices using sTiles, Esmail Abdul Fattah, Hatem Ltaief, Havard Rue, David Keyes, arXiv preprint arXiv:2504.19171, 2025.
5. sTiles : An Accelerated Computational Framework for Sparse Factorizations of Structured Matrices, Esmail Abdul Fattah, Hatem Ltaief, Håvard Rue, David Keyes, ISC High Performance 2025 Research Paper Proceedings, 2025.

Contributions

- Reformulation of G^r in the LU+ SI framework.
- Parallelization via Nested Dissection and block splitting.
- Implementation in STF (Chameleon/StarPU) and experimental validation.

Perspectives

- Comparaison to related work.
- Adaptive splitting.
- Distributed memory + GPU implementation.
- Block compression via low-rank approximations.

THANK YOU! Questions?